

TSM-AdvDABA : LABORATORY 2 – DIVING DEEPER WITH NEO4J

MSE : DATA SCIENCE



Vignette tirée de l'album *Sur la piste du Marsupilami* de Franquin et Greg (scénario) et Batem (dessin). Éditions MARSU PRODUCTIONS avril 2012, tous droits réservés.

Etudiant :
DEVÈNES Florian
DEMONT Kilian

Professeurs :
PARSIN Marcelo
RUMLEY Sébastien

1 ID du groupe et informations utiles

1. Group ID : DevDemAdvDaBa26
2. Namespace : dev-dem-adv-daba-26
3. ID of the pod containing Neo4j : neo4j-fdc9d7cbf-5g2lr
4. Neo4j credentials : neo4j / test1234
5. ID of the pod or pods prove that the loading actually happened : dblp-loader-56w84
6. Link to Git Repo : https://github.com/kdemont/AdvDaBa_labo2

2 The steps you followed and how your solution works

2.1 Step 1 : Chargement local

Avant de déployer le tout sur Kubernetes, la première étape a consisté à valider tout le processus de chargement en local pour vérifier que l'environnement Docker, l'application Java, la base de données Neo4j et le loader DBLP fonctionnaient correctement ensemble.

Le labo initial est composé de deux services Docker principaux : un conteneur Neo4j et un conteneur applicatif Java/Maven. L'application Java se connecte à Neo4j via Bolt et s'occupe de lire le fichier d'entrée au format JSONL pour insérer les éléments correspondants dans la BD graphe.

Dans un premier temps, le programme d'exemple fourni avec le labo a été exécuté sans modifications importantes. Cette première exécution créait simplement un nœud de test dans Neo4j. Ce premier test nous a permis de valider le démarrage de Neo4j, la connexion entre Java et Neo4j, l'exécution du conteneur et la lecture d'un fichier JSONL local.

Une fois cette baseline validée, le programme d'exemple a été remplacé par un loader DBLP. Le modèle de graphe implémenté est le suivant :

```
(:AUTHOR {_id, name})-[:AUTHORED]->(:ARTICLE {_id, title})
(:ARTICLE)-[:CITES]->(:ARTICLE)
```

Chaque article du fichier JSONL est représenté par un nœud **ARTICLE**. Chaque auteur est représenté par un nœud **AUTHOR**. La relation **AUTHORED** relie un auteur à un article qu'il a écrit, la relation **CITES** relie un article à un autre article cité dans ses références.

Pour éviter les doublons, deux contraintes d'unicité ont été créées au démarrage du loader :

```
CREATE CONSTRAINT article_id IF NOT EXISTS
FOR (a:ARTICLE)
REQUIRE a._id IS UNIQUE;
```

```
CREATE CONSTRAINT author_id IF NOT EXISTS
FOR (a:AUTHOR)
REQUIRE a._id IS UNIQUE;
```

Ces contraintes ont également l'effet de créer des index, et donc d'améliorer les performances des opérations **MATCH** et **MERGE**.

name	label	property	type	readCount
article_id	ARTICLE	_id	RANGE	...
author_id	AUTHOR	_id	RANGE	...

TABLE 1 – Index Neo4j créés par les contraintes d'unicité

Le chargement est effectué en deux passes. La première passe lit le fichier JSONL et crée les nœuds **ARTICLE**, les nœuds **AUTHOR** et les relations **AUTHORED**. La deuxième passe relit le fichier pour créer les

relations **CITES**. Cette approche en deux passes permet de garantir que les articles existent déjà dans la base avant de créer les relations de citation.

Le loader fonctionne avec un système de batchs. Au lieu d'envoyer une transaction Neo4j pour chaque ligne du fichier, les données sont regroupées puis envoyées à Neo4j avec des requêtes Cypher utilisant **UNWIND**. Cette approche réduit le nombre de transactions et améliore les performances du chargement.

2.1.1 Chargement du fichier test.jsonl

Le premier test de validation a été réalisé avec le petit fichier `test.jsonl` fourni dans la consigne du labo. Ce fichier a permis de vérifier que le loader était capable de lire une source JSONL distante et de transformer correctement les données en nœuds et relations Neo4j.

Le fichier contenait un très petit sous-ensemble des données DBLP. Le résultat obtenu lors de ce test est le suivant :

```
Articles loaded: 7
Authors loaded: 17
Total nodes loaded: 24
AUTHORED relationships loaded: 17
CITES relationships loaded: 1
```

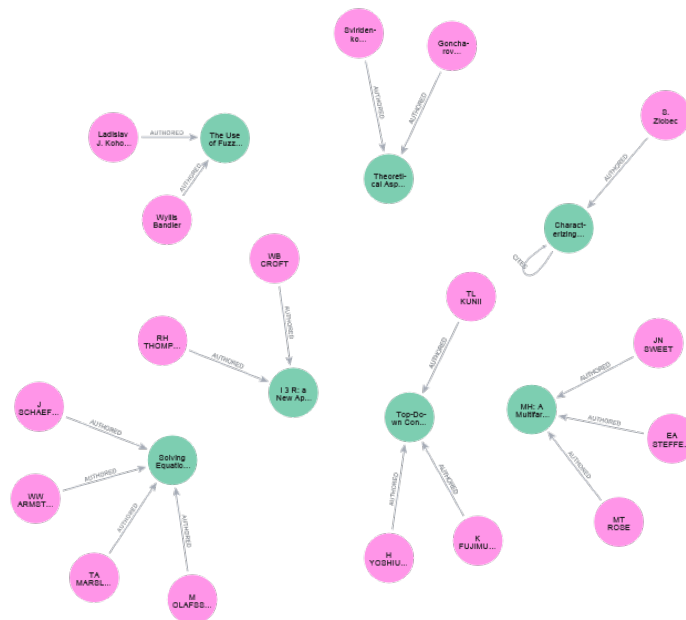


FIGURE 1 – Graph de l'échantillon de test

Ce test a confirmé que la structure principale du graphe était correctement créée. Le loader a bien généré les nœuds articles, les nœuds auteurs, les relations entre auteurs et articles, et les relations de citation.

À ce stade, l'objectif était pas d'évaluer les performances, mais de valider le bon fonctionnement général

du chargement. Comme les compteurs obtenus correspondaient aux attentes, le loader a ensuite pu être testé sur le fichier DBLP complet avec une limite progressive du nombre d'articles à charger.

2.1.2 Loading du fichier DBLP-Citation-network-V18.jsonl

Après la validation avec `test.jsonl`, l'étape suivante a consisté à tester le loader avec le dataset complet DBLP v18 :

`http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl`

pour éviter de charger l'ensemble du dataset, le loader a été configuré avec la variable `MAX_ARTICLES`. Cette variable permet de limiter le nombre d'articles lus depuis le fichier et donc de réaliser des tests progressifs.

Un premier test a été réalisé avec la configuration suivante :

```
JSON_FILE=http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl
MAX_ARTICLES=10000
BATCH_SIZE=1000
```

Avec cette configuration, le loader a lu le fichier DBLP complet en streaming et s'est arrêté après 10'000 articles. Les résultats obtenus sont les suivants¹ :

```
Input article lines read: 10000
Articles loaded: 10000
Authors loaded: 31062
Total nodes loaded: 41062
AUTHORED relationships loaded: 32373
CITES relationships loaded: 277
Duration seconds : 12
```

Ce premier test avec le vrai dataset a permis de valider la lecture depuis l'URL distante et la création correcte du graphe sur un volume plus important que le fichier de test.

Un second essai a ensuite été réalisé avec 1'000'000 articles. Le résultat obtenu est le suivant² :

```
Input article lines read: 1000000
Articles loaded: 1000000
Authors loaded: 1516347
Total nodes loaded: 2516347
AUTHORED relationships loaded: 3411998
CITES relationships loaded: 1616716
Duration seconds: 521
```

1. le log exact peut se retrouver dans le repo Github sous : `logs/local/run_10000.log`
2. le log exact peut se retrouver dans le repo Github sous : `logs/local/run_1M.log`

Ce test a montré que le loader était capable de traiter plusieurs millions d'éléments de graphe en local. Il a aussi servi de référence pour mesurer les optimisations réalisées par la suite.

Un test plus important a ensuite été effectué avec 4'000'000 articles³ :

```
Input article lines read: 4000000
Articles loaded: 4000000
Authors loaded: 3639993
Total nodes loaded: 7639993
AUTHORED relationships loaded: 13659841
CITES relationships loaded: 26018427
Duration seconds: 2957
```

Ce chargement a duré environ 49 minutes. Il a confirmé que le loader restait stable sur un volume important, mais il a également montré qu'il était nécessaire d'optimiser le temps de chargement avant l'exécution finale.

Après les optimisations, un chargement plus important a été réalisé avec 6'000'000 articles⁴ :

```
Input article lines read: 6000000
Articles loaded: 6000000
Authors loaded: 4590309
Total nodes loaded: 10590309
AUTHORED relationships loaded: 20494932
CITES relationships loaded: 58378898
Duration seconds: 4759
```

Ce dernier test a permis de dépasser les 10 millions de nœuds, ce qui répond à l'objectif du laboratoire. Au total, ce chargement a également créé plus de 78 millions de relations.

2.1.3 Tests en local et optimisations

Plusieurs tests locaux ont été réalisés pour améliorer les performances du loader avant le déploiement final.

Dans la première version fonctionnelle, les nœuds et les relations étaient créés avec l'instruction Cypher `MERGE`. Cette approche est sûre car elle évite de créer des doublons. Mais elle est plus coûteuse que `CREATE`, car Neo4j doit vérifier si l'élément existe déjà avant de le créer.

Pour les relations, la première version utilisait par exemple :

```
MERGE (author)-[:AUTHORED]->(article)
MERGE (source)-[:CITES]->(target)
```

3. le log exact peut se retrouver dans le repo Github sous : logs/local/run_4M.log

4. le log exact peut se retrouver dans le repo Github sous : logs/local/run_6M_optimised_batch5000.log

les essais sont réalisés sur une base Neo4j vide. Les relations n'existent pas encore au moment du chargement.

La première optimisation a consisté à remplacer **MERGE** par **CREATE** pour les relations :

```
CREATE (author)-[:AUTHORED]->(article)
CREATE (source)-[:CITES]->(target)
```

À ce stade, **MERGE** a été conservé pour les nœuds **AUTHOR**, car un même auteur peut apparaître dans plusieurs articles. L'utilisation de **CREATE** aurait entraîné une violation de la contrainte d'unicité en tentant de créer plusieurs occurrences d'un même auteur.

L'impact de cette optimisation a été mesuré sur le benchmark avec 1'000'000 articles :

Métrique	Avant optimisation	Après optimisation
Articles lus	1'000'000	1'000'000
Articles chargés	1'000'000	1'000'000
Auteurs chargés	1'516'347	1'516'347
Total de nœuds	2'516'347	2'516'347
Relations AUTHORED	3'411'998	3'411'998
Relations CITES	1'616'716	1'616'716
Durée	521 s	487 s

Le graphe obtenu avant et après optimisation est identique mais la durée de chargement est passée de 521 secondes à 487 secondes. Un gain de 34 secondes qui représente une amélioration d'environ 6.5%.

Une deuxième optimisation a consisté à augmenter la taille des batchs de 1'000 à 5'000. L'objectif était de réduire le nombre de transactions envoyées à Neo4j. Avec des batchs plus grands, plus de lignes sont insérées dans une même transaction, ce qui diminue le coût lié à la gestion des transactions.

Grâce à cette modification, la durée du benchmark avec 1'000'000 articles est passée de 521 secondes à 446 secondes. Ça représente un gain de 75 secondes, soit environ 14.4% d'amélioration.

La stratégie finale de chargement local repose sur les principes suivants :

- lecture du fichier JSONL directement depuis l'URL distante ;
- chargement en deux passes ;
- création de contraintes d'unicité sur **ARTICLE._id** et **AUTHOR._id** ;
- insertion des données par batchs avec **UNWIND** ;
- utilisation de **MERGE** pour les nœuds pouvant réapparaître, comme les auteurs ;
- utilisation de **CREATE** pour les relations lorsque la base est vide ;
- augmentation de la taille des batchs pour réduire le nombre de transactions.

La principale limite de l'utilisation de **CREATE** pour les relations est que la base Neo4j doit être vide avant chaque chargement. Si le loader est relancé plusieurs fois sur la même base sans nettoyage, les relations risquent d'être créées en double ou des erreurs vont se produire à cause des contraintes d'unicité.

2.1.4 Autres tentatives d'optimisation

D'autres tentatives d'optimisation de l'écriture dans la base de données ont été explorées, notamment l'exécution en parallèle lorsque cela était possible.

Il ressort que la création des nœuds **ARTICLE**, ainsi que des relations **AUTHORED** et **CITES**, ne pose pas de problèmes de concurrence. En revanche, la création des nœuds **AUTHOR** peut en engendrer. En effet, l'utilisation de **MERGE** implique une lecture préalable afin de vérifier si un auteur existe déjà, puis sa création le cas échéant, ce qui peut entraîner des conflits en environnement concurrent.

L'approche par batches a été conservée avec une taille importante (5000 articles). À cela ont été ajoutés des sous-batches, dix fois plus petits (500 articles), afin de permettre leur exécution en parallèle.

Une première implémentation a été réalisée à l'aide du plugin APOC :

```
CALL apoc.periodic.iterate(  
  'UNWIND $rows AS row RETURN row',  
  'CREATE (a:ARTICLE {_id: row.id, title: row.title})',  
  {batchSize: $apocBatchSize, parallel: true, params: {rows: $rows}}  
)
```

Code trouvable sur : https://github.com/kdemont/AdvDaBa_labo2/blob/concurrence/src/main/java/mse/advDB/Example.java.

Une seconde approche, plus récente, repose sur l'utilisation de `CALL { ... } IN CONCURRENT TRANSACTIONS` :

```
UNWIND $rows AS row  
CALL { WITH row  
  CREATE (a:ARTICLE {_id: row.id, title: row.title})  
} IN CONCURRENT TRANSACTIONS OF $subBatch ROWS
```

Code trouvable sur : https://github.com/kdemont/AdvDaBa_labo2/blob/concurrence-v2/src/main/java/mse/advDB/Example.java.

Ces expérimentations ne se sont pas révélées particulièrement concluantes, les temps d'exécution étant similaires à ceux obtenus sans parallélisme. Cela met en évidence l'efficacité des optimisations déjà apportées par le traitement par batches et l'utilisation de **UNWIND**, qui est nativement bien optimisé par Neo4j. Également le parallélisme

3 déploiement sur K8s

Après avoir validé le fonctionnement du loader en local, le labo a été déployée sur Kubernetes. L'objectif de cette étape était de vérifier que le chargement pouvait être exécuté dans un environnement plus proche d'un déploiement réel, avec Neo4j exposé comme service Kubernetes et le loader lancé sous forme de Job.

Dans ce déploiement, l'application Java ne se connecte plus à `localhost`, mais au service Kubernetes Neo4j via le nom DNS interne :

```
NEO4J_IP=neo4j
```

Le loader est exécuté dans un pod séparé et communique avec Neo4j via Bolt sur le port 7687. Le fonctionnement général reste identique à celui validé en local : le programme attend que Neo4j soit disponible, crée les contraintes d'unicité, puis effectue le chargement en deux passes.

3.1 premier test avec K8s sur `test.jsonl`

Le premier test Kubernetes a été réalisé avec le fichier de test `test.jsonl`. L'objectif était de vérifier que tous les éléments du déploiement fonctionnaient correctement avant de lancer le chargement du fichier DBLP complet.

La configuration du job était similaire à celle utilisée en local, avec une limite de lecture définie par `MAX_NODES` et une taille de batch définie par `BATCH_SIZE`. Les paramètres sont les suivants :

```
JSON file: http://vmrum.isc.heia-fr.ch/files/test.jsonl
Neo4j IP: neo4j
Max articles to read: 10000
Batch size: 5000
```

Le loader commence par attendre que Neo4j soit prêt :

```
Waiting for Neo4j...
Connected to Neo4j.
Constraints created or already existing.
```

Cette étape est importante dans Kubernetes, car le pod du loader peut démarrer avant que le pod Neo4j ne soit complètement disponible. Le programme contient donc une boucle d'attente qui vérifie régulièrement la connectivité avec Neo4j avant de commencer le chargement.

Le chargement du fichier de test s'est ensuite déroulé correctement en deux passes :

```
Pass 1/2: loading ARTICLE, AUTHOR and AUTHORED...
Opening remote JSONL stream: http://vmrum.isc.heia-fr.ch/files/test.jsonl
Pass 2/2: loading CITES relationships...
Opening remote JSONL stream: http://vmrum.isc.heia-fr.ch/files/test.jsonl
```

Les résultats obtenus sont les suivants⁵ :

```
Duration seconds: 8
Input article lines read: 7
Articles loaded: 7
Authors loaded: 17
Total nodes loaded: 24
AUTHORED relationships loaded: 17
CITES relationships loaded: 1
```

Ce premier test a permis de confirmer que le déploiement Kubernetes fonctionnait correctement. Le pod du loader a pu résoudre le service `neo4j`, se connecter à la base, créer les contraintes et charger les données attendues. Le résultat correspond au test local, ce qui valide la cohérence entre les deux environnements.

3.2 passage sur le gros fichier DBLP-Citation-network-V18.jsonl avec k8s

Un premier test a été effectué avec une limite de 10'000 articles pour valider la lecture du vrai fichier DBLP dans Kubernetes sans lancer directement le chargement complet.

La configuration utilisée était :

```
JSON file: http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl
Neo4j IP: neo4j
Max articles to read: 10000
Batch size: 5000
```

Le chargement s'est terminé avec succès⁶ :

```
Duration seconds: 23
Input article lines read: 10000
Articles loaded: 10000
Authors loaded: 31062
Total nodes loaded: 41062
AUTHORED relationships loaded: 32390
CITES relationships loaded: 278
```

Ce test a confirmé que le loader pouvait lire le fichier DBLP complet depuis son URL distante et insérer les données dans Neo4j depuis un pod Kubernetes.

Une exécution plus importante a ensuite été lancée avec une limite de 6'000'000 articles⁷ :

-
- 5. log disponible dans le repo sous : `logs/k8s/k8s_loader_test_jsonl.log`
 - 6. log disponible dans le repo sous : `logs/k8s/k8s_loader_10000_big_dataset.log`
 - 7. log disponible dans le repo sous : `logs/k8s/k8s_loader_failed_premature_eof.log`

Max articles to read: 6000000
Batch size: 5000

Cette exécution a correctement progressé pendant la première passe. Les logs montrent que les articles ont été lus progressivement jusqu'à environ 3'700'000 articles :

```
Pass 1 progress - articles read: 3600000  
Pass 1 progress - articles read: 3650000  
Pass 1 progress - articles read: 3700000
```

Le chargement s'est interrompu à cause d'une erreur réseau :

```
java.io.IOException: Premature EOF
```

L'erreur s'est produite pendant la lecture du flux HTTP distant. Le job Kubernetes s'est donc terminé en échec après environ 31 minutes :

```
BUILD FAILURE  
Total time: 31:58 min  
Failed to execute goal ... Premature EOF
```

Cette erreur n'était pas liée à Neo4j ni au modèle de graphe, mais à la stabilité du flux HTTP utilisé pour lire le très gros fichier DBLP. Comme le fichier est lu directement depuis une URL distante, une coupure du flux provoquait l'arrêt complet du loader dans la première version.

3.3 problématiques rencontrées et optimisations supplémentaires réalisées sur K8s

Le passage à Kubernetes a mis en évidence plusieurs problématiques supplémentaires par rapport aux tests locaux. La première difficulté concerne le démarrage des pods. Dans Kubernetes, Neo4j et le loader sont lancés comme composants séparés. Il est donc possible que le job du loader démarre avant que Neo4j soit complètement prêt. Pour éviter une erreur de connexion au démarrage, le programme attend explicitement que la base soit disponible :

```
Waiting for Neo4j...Neo4j not ready yet.Waiting for Neo4j...Connected to Neo4j.
```

Cette boucle d'attente rend le job plus robuste, car il ne dépend pas d'un ordre de démarrage parfaitement synchronisé entre les pods. La deuxième difficulté concerne le téléchargement des dépendances Maven au démarrage du pod. Les logs montrent que Maven télécharge de nombreuses dépendances au début de chaque exécution. Cela ajoute du temps au lancement du job et rend l'exécution dépendante de la connexion au dépôt Maven central. Les logs affichent plusieurs avertissements Maven

Les logs Maven affichent également quelques avertissements. En particulier, la version de la dépendance `junit-jupiter` utilise `LATEST` ou `RELEASE`, ce qui est déconseillé, et la version du plugin `exec-maven-plugin` n'est pas explicitement définie.

Ces avertissements ne bloquent pas l'exécution, mais ils indiquent que le build n'est pas totalement reproductible. Une amélioration possible serait de fixer explicitement les versions des dépendances et plugins Maven, puis de construire une image Docker contenant déjà les dépendances nécessaires. Cela éviterait de re-télécharger les dépendances à chaque lancement du job.

La problématique principale rencontrée sur le gros fichier DBLP est liée au flux HTTP. La première version du loader ouvrait simplement le fichier distant avec un flux HTTP classique. Cette approche fonctionnait pour les petits fichiers, mais elle n'était pas assez robuste pour un fichier de plusieurs GO. Quand coupure réseau se produisait, le flux était interrompu et l'application échouait avec une erreur **Premature EOF**. Pour résoudre ce problème, une version améliorée du loader a été faite. Elle remplace la lecture HTTP classique par un flux HTTP découpé en chunks, capable de reprendre la lecture à partir du dernier byte lu.

Le loader utilise pour cela l'en-tête **HTTP Range** :

Range: bytes=startByte-endByte

La nouvelle version affiche alors :

```
Opening resumable remote JSONL stream:  
http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl
```

Quand une interruption réseau se produit, le loader ne s'arrête plus immédiatement. Il attend, puis reprend le téléchargement à partir de la position exacte où il s'était arrêté :

```
HTTP stream interrupted at byte 9886178059  
with IOException: Premature EOF.
```

```
Resumed HTTP stream from byte 9886178059  
HTTP stream recovered.  
Consecutive retries reset to 0.
```

Cette optimisation a permis de rendre le chargement beaucoup plus robuste. Dans les logs du nouveau test, plusieurs interruptions sont observées, mais le programme parvient à reprendre le flux à chaque fois⁸ :

```
HTTP stream interrupted at byte 11044573885  
Resumed HTTP stream from byte 11044573885HTTP stream recovered.
```

Le loader comptabilise le nombre de reprises HTTP, le nombre de chunks chargés et le temps passé à attendre avant les reprises :

8. logs disponible dans le repo sous : logs/k8s/k8s_loader_too_slow_http_cut.log et log-s/k8s/k8s_loader_too_slow_v3.log

```
HTTP retry countHTTP chunk countHTTP retry wait seconds
Effective loading seconds excluding HTTP retry waits
```

Ces compteurs permettent de distinguer le temps réellement passé à charger les données dans Neo4j du temps perdu à cause des interruptions réseau. Une autre optimisation réalisée lors des tests Kubernetes a été l'augmentation de la taille des batchs. La première grande exécution utilisait : Batch size : 5000

La version optimisée a ensuite été lancée avec : Batch size : 10000

L'objectif était de réduire le nombre total de transactions envoyées à Neo4j. Avec des batchs plus grands, chaque transaction insère plus d'éléments, ce qui diminue le coût lié à la gestion des transactions. La version optimisée du loader repose sur plusieurs améliorations :

- attente active de la disponibilité de Neo4j avant le début du chargement ;
- lecture du fichier DBLP directement depuis l'URL distante ;
- découpage du flux HTTP en chunks ;
- reprise automatique du téléchargement après une coupure réseau ;
- utilisation de l'en-tête HTTP **Range** pour reprendre à partir du dernier byte lu ;
- augmentation de la taille des batchs à 10'000 lignes ;
- affichage de compteurs de progression et de statistiques HTTP ;
- conservation du chargement en deux passes pour séparer la création des nœuds et celle des relations CITES.

Grâce à ces modifications, le loader est devenu capable de continuer son exécution malgré des interruptions répétées du flux HTTP. Les logs montrent que la première passe a pu progresser jusqu'à 6'000'000 articles malgré de nombreuses reprises réseau⁹ :

```
Pass 1 progress - articles read: 6000000Pass 2/2: loading CITES relationships...
```

Cette étape a donc permis de passer d'un loader fonctionnel mais fragile à une version beaucoup plus adaptée au traitement d'un très gros fichier distant dans un environnement Kubernetes.

Il est possible de voir dans les logs que le flux HTTP est constamment interrompu, ce qui rends l'opération très longue avec des jobs dépassant les 24h. Pour pallier à ce problème de flux HTTP interrompu nous avons fait le choix sur la solution finale de faire un streaming par batch de 16MB gardés en cache. Cela nous a permis de réduire les problématiques liées au flux HTTP.

En contre partie, on lance de nombreuses connexions avec le serveur.

9. log disponible dans le repo sous : logs/k8s/k8s_loader_too_slow_http_cut.log

http.request.uri contains "DBLP-Citation-network-V18.jsonl"						
No.	Time	Source	Destination	Protcl	Length	Info
321298	12:51:17...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
321363	12:51:19...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
322135	12:51:20...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
322175	12:51:21...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
322893	12:51:21...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
323166	12:51:24...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
324259	12:51:26...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
324319	12:51:27...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
324928	12:51:28...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
324947	12:51:29...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
325668	12:51:29...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
325720	12:51:32...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
326564	12:51:36...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
326625	12:51:38...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
327492	12:51:40...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
327550	12:51:43...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
328384	12:51:44...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
328410	12:51:45...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
329209	12:51:45...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
329249	12:51:47...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
330054	12:51:48...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
330075	12:51:50...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
330893	12:51:50...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
330909	12:51:51...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
331639	12:51:52...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
331662	12:51:53...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
332566	12:51:54...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content
332841	12:51:56...	192.168.1.104	160.98.47.134	HTTP	333	GET /files/DBLP-Citation-network-V18.jsonl HTTP/1.1
333866	12:52:02...	160.98.47.134	192.168.1.104	HTTP	725	HTTP/1.1 206 Partial Content

FIGURE 2 – Analyse réseau (en local) des requêtes au serveur contenant le fichier

On observe au début une requête toutes les 3s environ, ce qui a un coût non négligeable par rapport à garder une connexion ouverte et streamer.

4 Déploiement final réalisé sur k8s et résultats

4.1 The loading time, in seconds, it took to load $N + K$ nodes (N articles, K authors)

Au moment de l'écriture de ce rapport notre dernier déploiement sur k8s est toujours en cours. Nous ne pouvons donc pas donner de temps final pour un déploiement sur k8s mais le log pourra être disponible ici : lien vers le pod via l'interface rancher.

le log du loader que nous avons réalisé en local est le suivant¹⁰ :

```
app-1 | Pass 2/2: loading CITES relationships...
app-1 | Opening remote JSONL stream: http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.
app-1 | === DBLP LOADER END ===
app-1 | End time: 2026-04-26T18:30:14.177195208Z
app-1 | Duration seconds: 4759
app-1 | Input article lines read: 6000000
app-1 | Articles loaded: 6000000
app-1 | Authors loaded: 4590309
app-1 | Total nodes loaded: 10590309
app-1 | AUTHORED relationships loaded: 20494932
app-1 | CITES relationships loaded: 58378898
app-1 | [INFO] -----
app-1 | [INFO] BUILD SUCCESS
app-1 | [INFO] -----
app-1 | [INFO] Total time: 01:19 h
```

Comme il est possible de le voir à la fin du log nous avons pu charger 6'000'000 Articles et 4'590'309 Auteurs pour un total de 10'590'309 Noeuds.

4.2 explanation of how this time can be recovered from the logs

le temps du run local peut être retrouvé dans les logs disponibles dans le repo Github en suivant le lien ci-après : logs/local/run_6M_optimised_batch5000.log

Pour le Loader déployé sur k8s qui est encore en cours d'exécution au moment de l'écriture de ce rapport, vous pourrez avoir accès au logs du run directement via rancher en allant dans le pod **dblp-loader-56w84** du namespace **dev-dem-adv-daba-26** ou via le lien suivant : lien vers le pod.

10. log disponible dans le repo sous : logs/local/run_6M_optimised_batch5000.log